

Advanced Higher Computing Science

Software Design and Development using Python



Booklet 2 – Data Types and Structures

Section 2 – Data Types and Structures	3
Data Types	3
Description	3
Data Structure – Parallel 1-D Arrays	3
Description	3
Data Structure – Records and Arrays of Records	3
Description	3
Data Structure – Objects and Arrays of Objects	4
Description	4
Designing an Array of Objects	5
Implementing an Array of Objects in SQA-RL	6
Implementation of an Array of Objects in Python 3	7
Testing of an Array of Objects in Python 3	8
Data Structure – Objects and Arrays of Objects – Exercise	8
Polymorphism	9
Description	9
Designing Polymorphism	9
Testing Polymorphism in Python 3	15
Polymorphism – Exercise	16
Data Structure – Objects and Arrays of Objects – File Handling.	17
Implementation of reading Array of Objects from a file in SQA-RL	17
Implementation and Testing of reading Array of Objects from a file with Python 3	18
Data Structure – Objects and Arrays of Objects – File Handling – Exercise	19
Data Structure – 2-D Arrays	20
Description and Design	20
Implementing 2-D Arrays in SQA-RL	21
Implementation and Testing of 2-D Arrays in Python 3	21
Data Structure – 2-D Arrays – Exercise	23
Data Structure – 2-D Array of Records/ Objects	24



Description and Design	24
Implementing 2-D Array of Records/ Objects in SQA-RL	27
Implementing 2-D Array of Records/ Objects in Python	29
Data Structure – 2-D Array of Records/ Objects Task	32

Section 2 – Data Types and Structures

Data Types

Description

The data types used for Advanced Higher Computing Science are exactly the same as those used at National 5 and Higher Computing Science, namely *string*, *integer*, *real*, *character* and *Boolean*. It is worth remembering that Python 3 does not have a *character* data type.

Data Structure – Parallel 1-D Arrays

Description

Parallel 1-D Arrays were met at Higher Computing Science. As a reminder, they consist of a number of individual 1-D Arrays with connected information in each element. However, the connection is conceptual – it exists only for us. The programming language does not connect the data together in any meaningful manner.

For example, the three Parallel 1-D Arrays below are connected by our understanding, but the Python code to manipulate them requires careful thought.

<i>Index</i>	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>	<i>4</i>
nameArray	Mark	Erica	Norman	Ben	Toni
QPArray	56	76	34	89	23
CWArray	34	12	23	24	45

It may be worth revisiting your Higher Computing Science notes to remind yourself of how Parallel 1-D Arrays work.

Data Structure – Records and Arrays of Records

Description

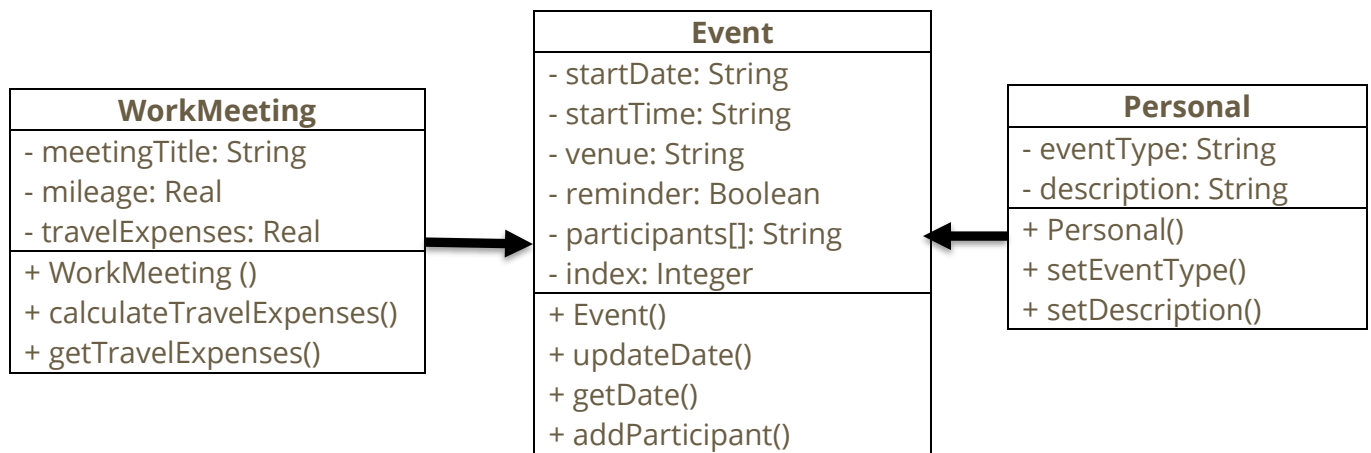
Similarly to 1-D Arrays, Records and Arrays of Records were covered as part of Higher Computing Science. As Python technically does not have a Record structure, Classes and Objects were used instead. This, in short, means that Records in Python are just Classes without any methods, meaning that the way Arrays of Objects work is exactly the same as in the next section, just without methods.

Data Structure – Objects and Arrays of Objects

Description

An Array of Objects is a single array containing 0, 1 or many instantiated Objects, build from a Class definition.

Recall the **Event** superclass, with subclasses **WorkMeeting** and **Personal**. The UML Class diagram is reproduced below for reference.



It is possible to create an Array of Objects of one of these classes.



Designing an Array of Objects

The design of an array of objects is not dissimilar to how we would design an array of record.

It comprises three parts:

1. The UML Class Diagram (see above)
2. The pseudocode to create the class and array, with a defined size.

Pseudocode for Data Structure

1. define the structure of a single Event record:
 - a. INTEGER id
 - b. STRING name
 - c. REAL salary
 - d. STRING department
 - e. STRING position
 - f. STRING hireDate
2. initialise results as an empty array of Event records (*size 1000*)

Implementing an Array of Objects in SQA-RL

The SQA Reference Language shown below can be used to show how an Array of Objects can be structured, specifically looking at lines 22 onwards.

```

Line 1    CLASS Event IS { STRING startDate, STRING startTime, STRING venue, BOOLEAN reminder,
            ARRAY OF STRING participants, INTEGER index }

Line 2        METHODS
Line 3        CONSTRUCTOR (STRING startDate, STRING startTime, STRING venue, BOOLEAN
            reminder )
Line 4            DECLARE THIS.startDate INITIALLY date
Line 5            DECLARE THIS.startTime INITIALLY startTime
Line 6            DECLARE THIS.venue INITIALLY venue
Line 7            DECLARE THIS.alert INITIALLY alert
Line 8            DECLARE THIS.participants INITIALLY [NULL] * 20
Line 9            DECLARE THIS.index INITIALLY 0
Line 10        END CONSTRUCTOR

Line 11        PROCEDURE updateDate (STRING eventDate)
Line 12            SET THIS.startDate TO eventDate
Line 13        END PROCEDURE

Line 14        FUNCTION getDate() RETURNS STRING
Line 15            RETURN THIS.startDate
Line 16        END FUNCTION

Line 17        PROCEDURE addParticipant(STRING name)
Line 18            SET THIS.participants[index] TO name
Line 19            SET index TO index + 1
Line 20        END PROCEDURE

Line 21    END CLASS

Line 22    DECLARE eventArray AS ARRAY OF Event INITIALLY [] * 20
Line 23    SET eventArray[0] TO ("14/04/22", "0900", "Main Office", True)
Line 24    SET eventArray[1] TO ("15/04/22", "1030", "Staff Room", True)

Line 25    eventArray[1].addParticipant("Erica Knowles")

```

Of course, it would be far more efficient to read the Objects into the Array of Objects from a file or database. Reading in from a file would work exactly the same way as in Higher Computing Science for an Array of Records. Reading in from a Database is covered later in this series of work.

Implementation of an Array of Objects in Python 3

The Python 3 code shown below can be used to show how an Array of Objects can be structured, specifically creating 2 instances of the Event Class and storing them in an array called event Array.

```
class Event:
    def __init__(self, startDate, startTime, venue, reminder):
        self.startDate = startDate #String
        self.startTime = startTime #String
        self.venue = venue #String
        self.reminder = reminder #Boolean
        self.participants = [""]*20 #Array of String
        self.index = 0 #Integer

    def updateDate(self, eventDate):
        self.startDate = eventDate

    def getDate(self):
        return self.startDate

    def addParticipant(self, name):
        self.participants[self.index] = name
        self.index = self.index + 1

#Creating and populating an Array of Objects
eventArray = [''] * 20
eventArray[0] = Event("14/04/2022", "0900", "Main Office", True )
eventArray[1] = Event("15/04/2022", "1030", "Staff Room", True )

eventArray[1].addParticipant("Erica Knowles")
print (eventArray[1].participants)
```

Program 10 – Creating an Array of Objects

Normally, to create an array of objects, the data would be read in from a data source, such as a text/ CSV file or a database.

Testing of an Array of Objects in Python 3

To ensure that the Array of Objects has been correctly created, 2 tests are carried out, as seen in the code above.

The first test is to ensure that any methods populate the correct object with data. This can be seen in the line of code `eventArray[1].addParticipant("Erica Knowles")`, which calls the `addParticipant()` method to apply "Erica Knowles" to the Event Object in the second position in the array (`eventArray[1]`). Note, this also places this data in the next position in the `participants` Array inside the object.

The second test is to ensure that properties are set correctly. This can be done using a simple print command to use either a method or property to display the contents of a few sample pieces of data. In the code on the previous page, `print (eventArray[1].participants)` displays the array of participants in the second object in the array of objects.

Data Structure – Objects and Arrays of Objects – Exercise

1. From the Classes, Properties and Methods Exercise created in Booklet 1, create an Array of Objects for the:
 - a. Vehicle Class. You should create at least three objects in the Array and provide evidence of using the methods to manipulate data in the Array of Objects.
 - b. Animal Class. You should create at least 5 Objects in the array and demonstrate evidence of the methods manipulating the data.
 - c. Computer Class. You should create at least 3 Objects in the array **through user input** and demonstrate evidence of the methods manipulating the data.

Polymorphism

Description

Polymorphism is a feature that allow multiple data types to behave the same way through a common interface.

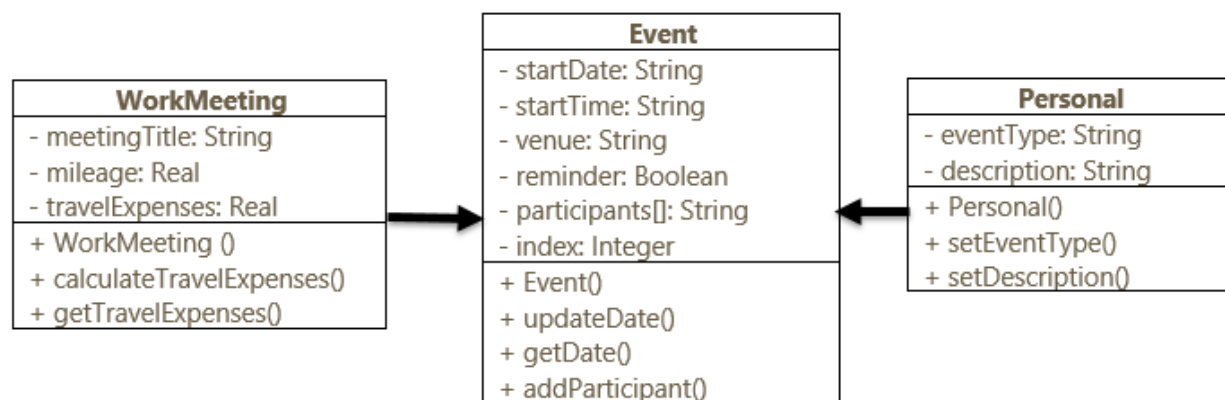
Definitions:

Polymorphism allows objects of a superclass and its subclasses to be stored in the same array, despite being of different data types.

Override is creating a method in a subclass with the same name but different behaviours to the method defined in the superclass.

Designing Polymorphism

Recall the **Event** superclass, with subclasses **WorkMeeting** and **Personal**. The UML Class diagram is reproduced below for reference.



WorkMeeting and **Personal** both inherit the properties and methods from the **Event** class.

This allows us to define an object of **WorkMeeting** that can, in certain circumstances, behave like an **Event**. Similarly, objects of the **Personal** class can behave like an **Event**.

This is really important when working with Arrays of Objects.

Consider the following Array of Event Objects.

	<i>Data Type</i>	<i>0</i>	<i>1</i>	<i>2</i>	<i>3</i>
eventArray	Event (Object)	14/04/22 0900 Main Office True [] 0	15/04/22 1030 Staff Room True [] 0	15/04/22 1100 Main Office True [] 0	15/04/22 1100 Staff Room True [] 0

and an instantiation of object of class Personal

```
personal1 = Person("16/04/22", "1500", "Staff Room", True, "Social", "Chat")
```

If we were to try to assign the personal1 Object into the next empty element of the array, what would happen, as it is not the same type of data as the rest of the eventArray?

In most cases, we cannot store different types of data in an array – for example, we could not store a mix of strings, integer and floats all in the same array.

The same should apply here – the **eventArray** is storing data of the type **Event** class, so should not be able to store data of type **Personal** class (or **WorkMeeting** class).

However, because the two subclasses are derived from the **Event** class, the array **will** be able to store the objects of subclasses of **Event** with some key limitations:

1. Only the properties of the superclass will be accessible via the array, so a **Personal** class object in the Array would not be able to access the eventType and description properties.
2. Only the methods of the superclass will be accessible via the array, so a **Personal** class object in the Array would not be able to access the setEventType() and setDescription() methods.
3. If the subclass has an overridden method from the superclass, the overridden method **will** be available through the array.

IMPORTANT NOTE 1

Although not really required for the course, it is worth identifying that we do not actually store Objects in variables or memory. Instead, a variable (or array element) is a *reference* to the Object in memory.

This means that the line of code above assigning `personal1` to `eventArray[4]` just means that both now refer to the same memory location that hold the Object. They just look at it differently – `personal1` sees it as a Personal Object, whereas `eventArray[4]` sees it as an Event Object.

As the Object can be viewed as different types depending what it "looking" at it, this is Polymorphism.

IMPORTANT NOTE 2

It is also worth identifying that Python does not actually use Arrays – instead it uses *lists* or *dictionaries*. Because of this, Python does not have the same limitation of ensuring that all data held in an array/ list structure is of the same type – making it very difficult to use Python to demonstrate this aspect of Object Oriented Programming.

Implementing Polymorphism in Python 3

```

class Event:
    def __init__(self, startDate, startTime, venue, reminder):
        self.startDate = startDate    #String
        self.startTime = startTime    #String
        self.venue = venue            #String
        self.reminder = reminder      #Boolean
        self.participants = [""]*20  #Array of String
        self.index = 0                #Integer

    def updateDate(self, eventDate):
        self.startDate = eventDate

    def getDate(self):
        return self.startDate

    def addParticipant(self, name):
        self.participants[self.index] = name
        self.index = self.index + 1

class WorkMeeting(Event):
    def __init__(self, startDate, startTime, venue, reminder, meetingTitle, mileage
    ):
        super().__init__(startDate, startTime, venue, reminder)
        self.meetingTitle = meetingTitle    #String
        self.mileage = mileage                #Real
        self.travelExpenses = 0              #Real

    def calculateTravelExpenses(self):
        self.travelExpenses = self.mileage * 1.5

    def getTravelExpenses(self):
        return self.travelExpenses

    def getDate(self):
        return ("The date of this Work Meeting is " + str(self.startDate))

class Personal(Event):
    def __init__(self, startDate, startTime, venue, reminder, eventType,
    description):
        super().__init__(startDate, startTime, venue, reminder)
        self.eventType = eventType          # String
        self.description = description      # String

    def setEventType(self, eventType):

```

```

        self.eventType = eventType

    def setDescription(self, description):
        self.description = description

    def getDescription(self):
        return self.description

# 1) instantiate objects of each class (top-level demo, runs without using __name__
guard)
e = Event("2026-04-23", "09:00", "Library", False)
w = WorkMeeting("2026-04-24", "10:00", "Office", True, "Budget Meeting", 12.0)
p = Personal("2026-04-25", "18:00", "Home", False, "Birthday", "Dinner with family")

# prepare list (array) of objects
events = [e, w, p]

# WorkMeeting overrides getDate(); all classes inherit addParticipant from Event
print("--- Call getDate() (shows override in WorkMeeting) and addParticipant
(inherited) ---")
for obj in events:
    print("getDate():", obj.getDate())
    obj.addParticipant("Alice")
    print("  first participant stored:", obj.participants[0])

# Demonstrate that subclass-only methods/properties must be accessed carefully.
print("\n--- Attempt to call WorkMeeting-only method getTravelExpenses() on every
item (runtime check) ---")
# calculate travel expenses for the WorkMeeting instance so it has a value
w.calculateTravelExpenses()
for obj in events:
    # This will succeed only for WorkMeeting; calling it on other instances will
    raise
    # AttributeError and crash the program – useful for demonstrating the difference.
    print("getTravelExpenses():", obj.getTravelExpenses())

print("\n--- Access Personal-specific properties/methods using isinstance (safe) ---")
for obj in events:
    print("Personal eventType:", obj.eventType)
    print("Personal description:", obj.getDescription())

```

Program 11 – Polymorphism



Testing Polymorphism in Python 3

To test to see if polymorphism has worked requires multiple objects, one of the superclass and at least one of each subclass. To show polymorphism is working means making sure that both behave differently when the overridden method is called, or that errors occur when trying to access subclass properties or methods.

```
--- Call getDate() (shows override in WorkMeeting) and addParticipant (inherited) ---
getDate(): 2026-04-23
    first participant stored: Alice
getDate(): The date of this Work Meeting is 2026-04-24
    first participant stored: Alice
getDate(): 2026-04-25
    first participant stored: Alice

--- Attempt to call WorkMeeting-only method getTravelExpenses() on every item (runtime check) ---
Traceback (most recent call last):
  File "c:\Users\gwr\poly.py", line 75, in <module>
    print("getTravelExpenses():", obj.getTravelExpenses())
                                ^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'Event' object has no attribute 'getTravelExpenses'
```


Polymorphism – Exercise

1. Create and test an example of Polymorphism on the Personal subclass of the Event superclass, using the updateDate() method to validate the date is in the correct format (dd/mm/yyyy). Use <https://www.tutorialspoint.com/How-to-do-date-validation-in-Python> for reference to date formats.

2. In the House, ForSale, ForRent classes example

- a. In the House class, add a method called getSummary() that returns a string in the following format:

```
[HOUSE] 12 High St, Edinburgh – Value: £250,000
```

- b. In the ForSale class, override the getSummary() method so that it returns a string in the following format:

```
[FOR SALE] 1 Market Rd, Glasgow – Asking: £190,000 | Sold: No
```

- c. In the ForRent class, override the getSummary() method so that it returns a string in the following format:

```
[FOR RENT] Flat 3, Stirling – Rent: £650/month | Rented: No
```

- d. Outside of the classes, write a function called printSummary(houseArray) that loops through the array and calls getSummary() on each object, printing the result.
- e. Call your function at the bottom of the program and check that the correct summary is printed for each object.
- f. Write a function called getTotalValue(houseArray) that loops through the array and returns the **total** of all houseValue attributes, regardless of what type of object each one is. Print the result in the following format:

```
Total portfolio value: £990,000
```

Data Structure – Objects and Arrays of Objects – File Handling.

Implementation of reading Array of Objects from a file in SQA-RL

The SQA Reference Language shown below can be used to show how an Array of Objects can be read from a file, specifically looking at lines 22 onwards.

```

Line 1      CLASS Event IS { STRING startDate, STRING startTime, STRING venue, BOOLEAN reminder,
                ARRAY OF STRING participants, INTEGER index }

Line 2      METHODS
Line 3      CONSTRUCTOR (STRING startDate, STRING startTime, STRING venue, BOOLEAN
                reminder )
Line 4      DECLARE THIS.startDate INITIALLY date
Line 5      DECLARE THIS.startTime INITIALLY startTime
Line 6      DECLARE THIS.venue INITIALLY venue
Line 7      DECLARE THIS.alert INITIALLY alert
Line 8      DECLARE THIS.participants INITIALLY [NULL] * 20
Line 9      DECLARE THIS.index INITIALLY 0
Line 10     END CONSTRUCTOR

Line 11     <Method definitions>

Line 21     END CLASS

Line 22     FUNCTION populateEventArrayFromFile() RETURNS Array OF Event
Line 23     DECLARE eA AS ARRAY OF STRING INITALLY []
Line 24     DECLARE entireFile AS STRING INITIALLY FROM OPEN "eventData.csv"
Line 25     DECLARE linesArray as STRING INITIALLY FROM entireFile.splitlines
Line 26     CLOSE "eventData.csv"
Line 27     FOR EACH line FROM linesArray DO
Line 28         DEFINE lineSplit AS <split line on comma>
Line 29         DEFINE currentEvent INITALLY Event (lineSplit[0], lineSplit[1], lineSplit[2],
                lineSplit[3])
Line 30     NEXT
Line 31     RETURN eA
Line 32     END FUNCTION
Line 33
Line 34     DEFINE eventArray AS ARRAY OF Event INITIALLY populateEventArrayFromFile()
  
```

The following points should be noted:

- It is important to examine the structure of the CSV file in the first instance to make sensible decisions about the data being read in for each line.

- Lines 27 – 30 (The fixed loop) go through each line from the CSV in turn and split into the individual data items, using the comma as the split. These are placed into the eA Array which is then returned to the top level program as the eventArray Array of Objects.
- The function populateEventArrayFromFile() is **not** a method of the Event Class – it is a separate function and hence is defined **after** the End Class. The indentation should represent this.

Implementation and Testing of reading Array of Objects from a file with Python 3

The code below is used to create the Event Class and read data from the attached eventData.csv file into an Array of Objects. This is then tested, very simply, in the last three lines by calling the populateEventArrayFromFile() function and pulling two pieces of data to the display to ensure the Array of Objects has been correctly populated.

```
class Event:
    def __init__(self, startDate, startTime, venue, reminder):
        self.startDate = startDate #String
        self.startTime = startTime #String
        self.venue = venue #String
        self.reminder = reminder #Boolean
        self.participants = [""]*20 #Array of String
        self.index = 0 #Integer

    def updateDate(self, eventDate):
        self.startDate = eventDate

    def getDate(self):
        return self.startDate

    def addParticipant(self, name):
        self.participants[self.index] = name
        self.index = self.index + 1

def populateEventArrayFromFile():
    eA = []
    entireFile = open("eventData.csv", "r")
    linesArray = entireFile.read().splitlines()
    entireFile.close()
    for line in linesArray:
        lineSplit = line.split(",")
        currentEvent = Event(lineSplit[0], lineSplit[1], lineSplit[2], lineSplit[3])
        eA.append(currentEvent)
    return eA
```

```
#Creating and populating an Array of Objects
eventArray = populateEventArrayFromFile()
print (eventArray[0].venue)
print (eventArray[23].startTime)
```

Program 12 - Read from CSV to Array of Objects

Data Structure – Objects and Arrays of Objects – File Handling – Exercise

Using the Vehicle Class, read the data from the supplied vehicleData.csv and provide a suitable systematic and comprehensive set of testing to show that your code works.

Data Structure – 2-D Arrays

Description and Design

A one-dimensional (1-D) array can be seen as data elements organised in a row. A two-dimensional (2-D) array is similar to a one-dimensional array, but it can be visualised as a grid (or table) with rows and columns.¹

For example, a nine-by-nine grid could be referenced with numbers for each row and letters for each column. A six-by-nine, two-dimensional array could be declared with a statement such as:

```
game [5][8]
```

This would give, in our mind, a grid like the one shown below, where each cell can be identified by the [row] [column] combination (or the [column] [row] combination).

game	1	2	3	4	5	6	7	8	9
1									
2									
3									
4								P	
5									
6									

For example, in the diagram above, the X is in row 3, column 7. This would be referred to as `game [3] [7]`.

(Bear in mind, indexing still starts at 0...)

As this is an array, each element in the array must be of the same data type. If a mix of data types is required, an Array of Records or an Array of Objects should be used.

¹ <https://www.bbc.co.uk/bitesize/guides/z4tf9j6/revision/3>

Implementing 2-D Arrays in SQA-RL

The SQA Reference Language shown below can be used to show how a 2-D Array can be created and populated. This sets the initial start state of a game where P represents a Player and E represents an Enemy.

```

Line 1    DECLARE game AS ARRAY OF ARRAY OF STRING INITALLY []
Line 2    SET game TO [ [ ] ]
Line 3    FOR row FROM 0 to 6 DO
Line 4        FOR column FROM 0 TO 8 DO
Line 5            SET game [row][column] TO " "
Line 6        END FOR
Line 7    END FOR

Line 8    SET game [3][7] TO "P"
Line 9    SET game [2][3] TO "E"
Line 10   SET game [5][2] TO "E"

Line 11   FOR row FROM 0 to 6 DO
Line 12       FOR column FROM 0 TO 8 DO
Line 13         SEND game [row][column] TO DISPLAY
Line 14       END FOR
Line 15   END FOR

```

The SQA Reference Language code above shows two instances of the nested loop concept – a very important piece of thinking when using 2-D Arrays. In short, the inner loop cycles through each item in a row, with the outer loop cycling through the rows.²

Implementation and Testing of 2-D Arrays in Python 3

Making use of external Python libraries can significantly ease the coding required to implement a 2-D Array, particularly when it comes to displaying output. The code presented uses the NumPy and Pandas libraries. These are beyond the scope of the Advanced Higher Computing Science Course but make working with the data in a table format very easy.

² <https://www.youtube.com/watch?v=B5GhIXhDfoE&t=25s>

```

import numpy, pandas #numpy and pandas provide useful functions for the display and
manipulation of 2-d Arrays

rows, cols = (6,9) #define the number of rows and columns in the 2-D Array
game = [" " for i in range(cols+1)] for j in range(rows+1)] #Create the empty 2-D
Array using a nested loop. Note the use of +1 to account for 0th element.

#used to set a specific element in an array
game[3][7] = "P"
game[2][3] = "E"
game[5][2] = "E"

#the following sections of code use numpy and pandas imported modules to ease display
of 2-D Arrays.
rowHeaders = ["1","2","3","4","5","6"] #creates the row headers for the display
colHeaders = ["1","2","3","4","5","6","7","8","9"] # creates the column headers for
the display
dispGame = numpy.array(game)
dispGame = pandas.DataFrame(dispGame[1:,1:], columns = colHeaders, index =
rowHeaders)
dispGame.columns.name = "Game"

print (dispGame)

```

Program 13 – Defining a 2D Array

Points to note are as follows:

- The two libraries, numpy and pandas are imported at the start of the program.
- The size of the 2-D Array is then defined.
- A nested loop is used in a single line of code to populate each element in the array with a space " " character. The +1 is used to account for the 0th element.
- i and j are common shortcut variables for controlling the nested loop. x and y are also often used, but these can be confused with co-ordinates.
- The row and column headers are given as simple 1-D Arrays of string because the data in the game 2-D Array is also string.

Data Structure – 2-D Arrays – Exercise

1. A coach company runs a daily journey from Edinburgh to London. The coach has 13 rows of 4 seats that passengers can book. There is a toilet (T) taking up the space of 4 seats, so these are not available for booking. Some seats have already been booked (B).³

	1	2	3	4
1				
2				
3		B		
4				
5			T	T
6			T	T
7				
8				B
9				
10	B			
11				
12				
13				

- a. Design, implement and test a 2-D Array that will create the start state of the coach as shown above.
- b. Design, implement and test a function/ procedure that will ask a passenger for the row and column they wish to sit in. Their entry should be validated (1-4 and 1-13).

³ Adapted from https://www.sqa.org.uk/pastpapers/papers/papers/2019/NAH_Computing-Science_QP_2019.pdf

The program will check if the seat is available. If it is, it will record this with a B. If not, it will ask the passenger to try again.

Data Structure – 2-D Array of Records/ Objects

Description and Design

As each element in a 2D Array has to hold the same data type, it is possible for each element to hold a record or an object.

For example, each seat in a cinema could be a **record** holding four fields.

The seats are arranged as a **2D array** — indexed by `cinema[row][seat]` — mirroring the real-world layout of a screening room.

First we design the Class and the 2D Array:

Seat
- row: integer - seat_num: integer - availability: String - status: string - price: real
+ Seat() + book() + is_available()

cinema	1	2	3	4	5	6
1	R0 S0 available 8.50	R0 S1 available 8.50	R0 S2 available 8.50	R0 S3 available 8.50	R0 S4 available 8.50	R0 S5 available 8.50
2	R1 S0 available 12.00	R1 S1 booked 12.00	R1 S2 available 12.00	R1 S3 available 12.00	R1 S4 available 12.00	R1 S5 available 12.00
3	R2 S0 booked 12.00	R2 S1 available 12.00	R2 S2 available 12.00	R2 S3 available 12.00	R2 S4 available 12.00	R2 S5 available 12.00



4

R2 S0 available 15.00	R2 S1 available 15.00	R2 S2 available 15.00	R2 S3 available 15.00	R2 S4 booked 15.00	R2 S5 available 15.00
--------------------------------	--------------------------------	--------------------------------	--------------------------------	-----------------------------	--------------------------------



Second, we design the methods using pseudocode:

To book a seat (book method)

1. Check if the seat has already been booked.
2. If yes,
 - a. tell the user.
3. Otherwise
 - a. mark the seat as booked
 - b. confirm the booking with the price.

Check seat availability (is_available method)

1. Report whether the seat is currently available.

Finally, we design the program functionality:

Create a cinema function

1. Start with an empty cinema.
2. For each row,
 - a. For each seat position.
 - i. Create a new Seat.
 - b. Add the completed row to the cinema.

Book a seat procedure

1. Find the correct Seat object in the cinema using the row and seat number,
2. Call the book method.

Display the cinema procedure

1. Create a blank grid.
2. Visit every seat and place its text summary into the matching position in the grid.
3. Attach row and seat labels to the grid.
4. Print it out as a neatly formatted table.

Count available seats function

1. Start a running total at zero.
2. For each row,
 - a. For each seat position.
 - i. Call is_available method.
 - ii. if is_available
 1. increment running total

Revenue Report function

1. Start a running total at zero.
2. For each row,
 - a. For each seat position.
 - i. Call is_available method.
 - ii. if is_available
 1. add price to running total

Implementing 2-D Array of Records/ Objects in SQA-RL

The SQA Reference Language shown below can be used to represent the implementation.

```

Line 1    CLASS Seat IS { INTEGER row, INTEGER seat_num, STRING is_available, REAL price}
Line 2    METHODS
Line 3    CONSTRUCTOR (INTEGER row, INTEGER seat_num, STRING is_available, REAL
            price )
Line 4    DECLARE THIS.row INITIALLY row
Line 5    DECLARE THIS.seat_num INITIALLY seat_num
Line 6    DECLARE THIS.is_available INITIALLY is_available
Line 7    DECLARE THIS.price INITIALLY price
Line 8    END CONSTRUCTOR

Line 9    FUNCTION book () RETURNS BOOLEAN
Line 10    IF THIS.status = "booked" THEN
Line 11        SEND "This seat is Booked" TO DISPLAY
Line 12        RETURN False
Line 13    ELSE
Line 14        THIS.status = "booked"
Line 15        SEND "You have booked this seat" TO DISPLAY
Line 16        RETURN True
Line 17    END FUNCTION

Line 18    FUNCTION is_available() RETURNS STRING
Line 19        RETURN THIS.status
Line 20    END FUNCTION
Line 21    END CLASS
  
```

```

Line 22    FUNCTION create_cinema (rows, seats_per_row) RETURNS ARRAY of Seat
Line 23        FOR r FROM 1 TO rows DO
Line 24            FOR s FROM 1 TO seat_per_row DO
Line 25                SET cinema[r][s] TO Seat(r,s)
Line 26            END FOR
Line 27        END FOR
Line 28        RETURN cinema
Line 29    END FUNCTION

Line 30    PROCEDURE book_seat (cinema, row, seat_num)
Line 31        cinema[row][seat_num].book()
Line 32    END PROCEDURE

Line 33    PROCEDURE display_cinema(cinema, rows, seats_per_row)
Line 34        FOR r FROM 1 TO rows DO
Line 35            FOR s FROM 1 TO seat_per_row DO
Line 36                SET cinema[r][s] TO Seat(r,s)
Line 37            END FOR
Line 38        END FOR
Line 39    END PROCEDURE

Line 40    FUNCTION count_available(cinema, rows, seats_per_row) RETURNS INTEGER
Line 41        SET count TO 0
Line 42        FOR r FROM 1 TO rows DO
Line 43            FOR s FROM 1 TO seat_per_row DO
Line 44                IF cinema[r][s].is_available() THEN
Line 45                    SET count TO count +1
Line 46                END FOR
Line 47            END FOR
Line 48        RETURN count
Line 49    END FUNCTION

Line 50    FUNCTION count_available(cinema, rows, seats_per_row) RETURNS INTEGER
Line 51        SET total TO 0.0
Line 52        FOR r FROM 1 TO rows DO
Line 53            FOR s FROM 1 TO seat_per_row DO
Line 54                IF NOT cinema[r][s].is_available() THEN
Line 55                    SET total TO total + cinema[r][s].price
Line 56                END FOR
Line 57            END FOR
Line 58        RETURN count
Line 59    END FUNCTION

```

Implementing 2-D Array of Records/ Objects in Python

The Python code shown below can be used to represent the implementation.

```
import numpy
import pandas

ROWS = 4
SEATS_PER_ROW = 6
ROW_PRICES = {0: 8.50, 1: 12.00, 2: 12.00, 3: 15.00}

# — Seat Class —————
class Seat:
    def __init__(self, row, seat_num):
        self.row = row
        self.seat_num = seat_num
        self.status = "available"
        self.price = ROW_PRICES[row]

    def book(self):
        if self.status == "booked":
            print(f" Seat {self.row}/{self.seat_num} is already taken.")
            return False
        self.status = "booked"
        print(f" Booked seat {self.row}/{self.seat_num} – £{self.price:.2f}")
        return True

    def is_available(self):
        return self.status == "available"

    def __str__(self):
        return (f"R:{self.row} S:{self.seat_num}\n"
                f"£{self.price:.2f}\n"
                f"{self.status}")

# — Cinema functions —————
def create_cinema(rows, seats_per_row):
    cinema = []
    for r in range(rows):
        row = []
        for s in range(seats_per_row):
            row.append(Seat(r, s))
        cinema.append(row)
    return cinema
```

```

def book_seat(cinema, row, seat_num):
    cinema[row][seat_num].book()

def display_cinema(cinema):
    grid = [[" " for s in range(SEATS_PER_ROW + 1)] for r in range(ROWS + 1)]

    for r in range(ROWS):
        for s in range(SEATS_PER_ROW):
            grid[r + 1][s + 1] = str(cinema[r][s])    # use __str__ for full details

    rowHeaders = [f"Row {r}" for r in range(ROWS)]
    colHeaders = [f"Seat {s}" for s in range(SEATS_PER_ROW)]

    dispCinema = numpy.array(grid)
    dispCinema = pandas.DataFrame(dispCinema[1:, 1:], columns=colHeaders,
index=rowHeaders)
    dispCinema.columns.name = "Cinema"

    print("\n      --- SCREEN ---\n")
    print(dispCinema)
    print()

def count_available(cinema):
    count = 0
    for r in range(ROWS):
        for s in range(SEATS_PER_ROW):
            if cinema[r][s].is_available():
                count += 1
    return count

def revenue_report(cinema):
    total = 0.0
    for r in range(ROWS):
        for s in range(SEATS_PER_ROW):
            if not cinema[r][s].is_available():
                total += cinema[r][s].price
    return total

# — Demo —————
cinema = create_cinema(ROWS, SEATS_PER_ROW)

print("=== Initial seating plan ===")
display_cinema(cinema)

print("=== Booking seats ===")
book_seat(cinema, 0, 1)

```

```

book_seat(cinema, 1, 2)
book_seat(cinema, 2, 0)
book_seat(cinema, 3, 4)
book_seat(cinema, 0, 1) # already booked – should fail

print("\n=== Updated seating plan ===")
display_cinema(cinema)

print(f"Available seats : {count_available(cinema)}")
print(f"Revenue so far : £{revenue_report(cinema):.2f}")

```

Program 14 - 2D Array of Objects

Points to note are as follows:

1. The line of code below is a Python dictionary,

```
ROW_PRICES = {0: 8.50, 1: 12.00, 2: 12.00, 3: 15.00}
```

used to map the row number (0,1,2,3) to a value for the price (8.50, 12.00, 12.00 and 15.00). This makes it very easy to change prices per row later, if needed. It is used in the Class Constructor to allocate a price based on the row of the current Seat object being instantiated.

```
self.price = ROW_PRICES[row]
```


Data Structure – 2-D Array of Records/ Objects Task

A program is required to manage a greenhouse, laid out in a 10 by 5 grid of squares.

Each square in the grid will contain a plant.

1. Functional Requirements

Grid Population: The system must initialize a 2D array (10x5) of Plant objects

Object Attributes: Each Plant object must store:

- SpeciesName (e.g., "Tomato", "Fern").

- HealthScore (Integer 0-100).

- NeedsWater (Boolean).

- LastFed (String/Date).

Inventory Update: The system must allow users to "plant" a new object into an empty null/placeholder slot in the array.

Maintenance Logic: The system must be able to iterate through the entire grid and trigger a water() method for every plant where NeedsWater is true.

Health Audit: The system must calculate the average HealthScore of all plants in a specific row (section).

Search Functionality: The system must locate a plant by name and return its coordinates (x, y) in the grid.

2. End-User Requirements

Greenhouse Overview: The user must be able to see a "Map" of the nursery, displaying the species name in each grid coordinate.

Care Alerts: The user should see a highlighted list or visual marker (e.g., an asterisk *) for any plant that currently needs water.

Planting Interface: The user must be able to select a "Plot" (row/column) to add a new plant's details.

Status Report: The user should be able to click/select a specific plot to view a full "Care Label" (all object properties) for that plant.

Safety Checks: The user must be notified if they try to "plant" something in a spot that is already occupied.